SPRINT 8 · DUELO DE MODELOS · S08.1

Árboles de decisión

TurisData ya sabe que se pueden predecir las cancelaciones. Ahora: ¿qué modelo ponemos en producción?

El reto: un duelo con tres criterios

> «¿El más preciso? ¿El que yo pueda entender? ¿El más barato de mantener?»

Rendimiento

AUC y *average precision* en validación cruzada.

Explicabilidad

¿Puede el gerente entender por qué se marca una reserva?

Coste

Tiempo de ajuste, de predicción y tamaño del modelo.

El **leaderboard** mide a todos los modelos igual (cuaderno `S08_06`). Hoy: árboles y *ensembles*. Mañana: SVM y búsqueda de hiperparámetros.

La idea: preguntas de sí/no encadenadas

- El árbol busca, entre todas las variables y umbrales, **la pregunta que mejor separa** canceladas de no canceladas (índice de **Gini** o entropía: separación más «pura» = mejor).
- Repite el proceso en cada rama hasta llegar a **hojas**.

```
arbol_peq.tree_.feature[0]          # índice de la variable de la raíz (Ejercicio 1)
variable_raiz = nombres[arbol_peq.tree_.feature[0]]
n_hojas = arbol_peq.get_n_leaves()
```

No necesita **escalar** las variables y cada predicción se puede explicar con un camino de preguntas.

Sobreajuste: el árbol libre

```
arbol_libre = DecisionTreeClassifier(random_state=SEMILLA).fit(X_train, y_train)
auc_train_libre = roc_auc_score(y_train, arbol_libre.predict_proba(X_train)[:, 1])
auc_test_libre  = roc_auc_score(y_test,  arbol_libre.predict_proba(X_test)[:, 1])
```

- Sin límites, sigue preguntando hasta que **cada hoja es pura**: memoriza el entrenamiento (AUC 1,0 en train) incluido el ruido.
- La señal: **gran brecha** entre train y test (Ejercicio 2).

Poda: frenar el crecimiento

Hiperparámetro	Qué limita
`max_depth`	nº de preguntas seguidas
`min_samples_leaf`	mínimo de reservas por hoja (evita hojas «de 2 casos»)
`ccp_alpha`	poda **a posteriori**: recorta ramas que no compensan

```
mejor_min_hoja = hojas_min[int(np.argmax(auc_cv_hojas))]                # Ejercicio 3
arbol_podado = DecisionTreeClassifier(min_samples_leaf=mejor_min_hoja,
                                      random_state=SEMILLA).fit(X_train, y_train)
```

Importancia de variables… y su gran defecto

```
importancias = pd.Series(arbol_podado.feature_importances_, index=nombres).sort_values(ascending=False)
top3 = importancias.head(3).index.tolist(); suma_top3 = importancias.head(3).sum()   # Ejercicio 4
```

- `feature_importances_` suma 1; describe lo que mira **el modelo**, no una causa.
- Tiende a inflar las numéricas con muchos valores distintos.
- **Inestable:** con submuestras del 80 % de train salen 5 árboles que predicen distinto las mismas reservas de test. Es lo que arreglan los *ensembles*.

Mini-reto: reglas para el gerente

```
arbol_gerente = DecisionTreeClassifier(max_depth=3, min_samples_leaf=50,
                                       random_state=SEMILLA).fit(X_train, y_train)
print(export_text(arbol_gerente, feature_names=nombres))
```

Escribe en la lista `reglas` **tres frases** en lenguaje de negocio a partir del árbol: «Si la tarifa es no reembolsable y la antelación es corta, el riesgo de cancelación es bajo».

Pregunta para pensar: si el gerente pregunta «¿por qué se marcó **esta** reserva?», ¿qué ventaja tiene un árbol frente a un modelo de 300 árboles?

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S08_01_arboles_decision — 35 min

Árbol de decisión: reglas, poda e importancia

sprints/S08/notebooks/S08_01_arboles_decision.ipynb